CODE :-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node {
   int PRN;
   char name[50];
   struct node *next;
};

struct node *head = NULL;

struct node *createNode() {
   struct node *newNode;
   newNode = (struct node *)malloc(sizeof(struct node));

   if (newNode == NULL) {
      printf("Memory allocation failed\n");
      exit(1);
   }

   printf("Enter PRN: ");
   scanf("%d", &newNode->PRN);

   printf("Enter Name: ");
   scanf(" %49[^\n]", newNode->name);

   newNode->next = NULL;
   return newNode;
```

```
}

void insertBeginning() {
    struct node *newNode = createNode();
    newNode->next = head;
    head = newNode;
    printf("Member added successfully\n");
}

void insertEnd() {
    struct node *newNode = createNode();
    struct node *temp;

    if (head == NULL) {
        head = newNode;
    } else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }

    printf("Member added successfully\n");
}

void deleteMember() {
    struct node *temp = head, *prev = NULL;
    int prn;

    printf("Enter PRN to delete: ");
    scanf("%d", &prn);
```

```
    while (temp != NULL && temp->PRN != prn) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Student not found\n");
        return;
    }

    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;

    free(temp);
    printf("Member deleted successfully\n");
}

void display() {
    struct node *temp = head;

    if (head == NULL) {
        printf("No students registered\n");
        return;
    }

    while (temp != NULL) {
        printf("PRN: %d, Name: %s\n", temp->PRN, temp->name);
        temp = temp->next;
```

```
    }
}

void countMembers() {
    struct node *temp = head;
    int count = 0;

    while (temp != NULL) {
        count++;
        temp = temp->next;
    }

    printf("Total members: %d\n", count);
}

void sortList() {
    struct node *p, *q;
    int prn;
    char name[50];

    for (p = head; p != NULL; p = p->next) {
        for (q = p->next; q != NULL; q = q->next) {
            if (p->PRN > q->PRN) {
                prn = p->PRN;
                p->PRN = q->PRN;
                q->PRN = prn;

                strcpy(name, p->name);
                strcpy(p->name, q->name);
                strcpy(q->name, name);
            }
```

```
        }
    }

    printf("List sorted by PRN\n");
}

void reverseList() {
    struct node *prev = NULL, *current = head, *next;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }

    head = prev;
    printf("List reversed successfully\n");
}

void mergeList() {
    struct node *head2 = NULL, *temp;
    int n, i;

    printf("Enter number of members in second list: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        struct node *newNode = createNode();

        if (head2 == NULL) {
```

```
            head2 = newNode;
        } else {
            temp = head2;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newNode;
        }
    }

    if (head == NULL) {
        head = head2;
    } else {
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = head2;
    }

    printf("Lists merged successfully\n");
}

int main() {
    int choice;

    do {
        printf("\n--- Pinnacle Club Management ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Delete Member\n");
        printf("4. Display Members\n");
        printf("5. Count Members\n");
```

```
        printf("6. Sort by PRN\n");
        printf("7. Reverse List\n");
        printf("8. Merge Lists\n");
        printf("9. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: insertBeginning(); break;
            case 2: insertEnd(); break;
            case 3: deleteMember(); break;
            case 4: display(); break;
            case 5: countMembers(); break;
            case 6: sortList(); break;
            case 7: reverseList(); break;
            case 8: mergeList(); break;
            case 9: printf("Exiting program\n"); break;
            default: printf("Invalid choice\n");
        }
    } while (choice != 9);

    return 0;
}
```

Output :-

```
--- Pinnacle Club Management ---
1. Insert at Beginning
2. Insert at End
3. Delete Member
4. Display Members
5. Count Members
```

6. Sort by PRN

7. Reverse List

8. Merge Lists

9. Exit

Enter choice: 1

Enter PRN: 101

Enter Name: Rahul

Member added successfully

--- Pinnacle Club Management ---

Enter choice: 2

Enter PRN: 103

Enter Name: Priya

Member added successfully

--- Pinnacle Club Management ---

Enter choice: 1

Enter PRN: 102

Enter Name: Amit

Member added successfully

--- Pinnacle Club Management ---

Enter choice: 4

PRN: 102, Name: Amit

PRN: 101, Name: Rahul

PRN: 103, Name: Priya

--- Pinnacle Club Management ---

Enter choice: 5

Total members: 3

--- Pinnacle Club Management ---

Enter choice: 6

List sorted by PRN

--- Pinnacle Club Management ---

Enter choice: 4

PRN: 101, Name: Rahul

PRN: 102, Name: Amit

PRN: 103, Name: Priya

--- Pinnacle Club Management ---

Enter choice: 7

List reversed successfully

--- Pinnacle Club Management ---

Enter choice: 4

PRN: 103, Name: Priya

PRN: 102, Name: Amit

PRN: 101, Name: Rahul

--- Pinnacle Club Management ---

Enter choice: 3

Enter PRN to delete: 102

Member deleted successfully

--- Pinnacle Club Management ---

Enter choice: 4

PRN: 103, Name: Priya

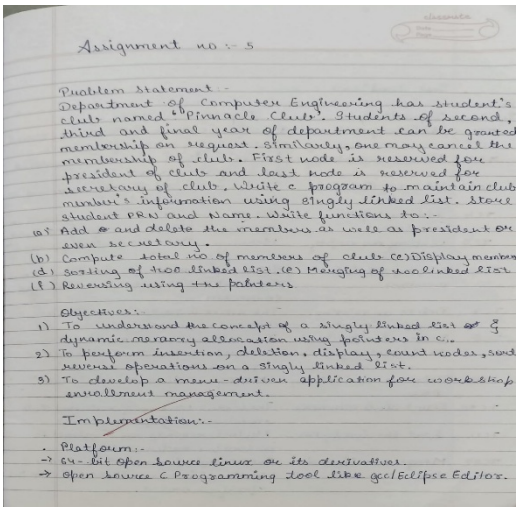
PRN: 101, Name: Rahul

--- Pinnacle Club Management ---

Enter choice: 9

Exiting program

WRITEUP :-



Assignment no :- 5

Problem Statement :-
Department of Computer Engineering has student's club named "Pinnacle Club". Students of second, third and final year of department can be granted membership on request. Similarly, one may cancel the membership of club. First node is reserved for president of club and last node is reserved for secretary of club. Write c program to maintain club member's information using singly linked list. Store student PRN and Name. Write functions to :-
(a) Add & and delete the members as well as president or even secretary.
(b) Compute total no of members of club (c) Display members
(d) sorting of the linked list. (e) Merging of two linked list
(f) Reversing using two pointers

Objectives :-
1) To understand the concept of a singly linked list & dynamic memory allocation using pointers in C.
2) To perform insertion, deletion, display, count nodes, sort, reverse operations on a singly linked list.
3) To develop a menu-driven application for workshop enrollment management.

Implementation :-

- Platform :-
→ 64-bit Open Source Linux or its derivatives.
→ Open Source C Programming tool like gcc/Eclipse Editor.

Test Cond":-

Test Case	Input/Output	Expected Output
TC1	Add the president	President's PRN and Name are stored successfully as the first node.
TC2	Add the secretary	Secretary PRN and Name are stored successfully as the last node.
TC3	Add a regular club member	Student record is added successfully to the linked list.
TC4	Add multiple members	All the students records are stored in the linked list in the specified order
TC5	Display an empty club list.	Message :- "No students registered"
TC6	Display all club members	All student records, including the President and Secretary, are displayed correctly.
TC7	Delete an existing regular member.	The selected student's record is deleted successfully.
TC8	Delete the President or Secretary.	The selected special member is removed successfully and the list/ head are updated correctly.
TC9	Delete a non-existing PRN.	Message :- "Student not found."
TC10	Count club members	The correct total no. of registered club members is displayed.
TC11	Sort the list by PRN.	Student records are displayed in ascending order of PRN.
TC12	Merge two linked list.	The records from both lists are combined into one linked list in the specified order.
TC13	Reverse the linked list using three pointers	The order of students record is reversed correctly
TC14	Delete all members and display	The list becomes empty and appropriate msg, such as "No students registered." is displ

Pseudo Code :-

S-1 Start
S-2 Create structure NODE :
PRN
Name
next
S-3 Set Head = Null
S-4 Repeat
Display menu :
1. Insert at Beginning
2. Insert at End
3. Delete by PRN
4. Display Members
5. Count Members
6. Sort by PRN
7. Reverse List
8. Merge two lists
9. Exit
S-5 Read Choice
S-6 Switch (choice)

case 1 :- Insert at Beginning
Create newNode
Read PRN and Name
Set newNode.next = head
Set head = newNode
Print "Member added successfully"
Break.

Case 2 :- Insert at end
Create newNode
Read PRN and Name
Set newNode.next = Null
If head == Null Then
Set head = newNode
Else
Set temp = head
While temp.next != Null
Set temp = temp.next
End While
Set temp.next = newNode
End If
Print "Member added successfully"
Break

Case 3 :- Delete by PRN.
Read PRN
If head == Null Then
Print "List is empty"
Else if head.PRN == PRN then
Set temp = head
Set head = head.next
Free temp
Print "Member deleted"
Else
Set temp = head
While temp.next != Null and temp.next.PRN != PRN
Set temp = temp.next
End While
If temp.next == Null then
Print "Student not found"
Else
Set del = temp.next
Set ~~temp.next~~ del = temp.next
Set temp.next = del.next
Free del
Print "Member deleted"
End if
End if
Break.

Case 4 :- Display Members
If head == Null then
Print "No students registered"
Else
Set temp = head
While temp != Null
Print temp.PRN, temp.Name
Set temp = temp.next
End While
End if
Break.

Case 5 :- Count Members.
Set Count = 0
Set temp = head
While temp != Null.
count = count + 1
Set temp = temp.next.
End While
Print Count
Break.

Case 6 :- Sort by PRN
Set ptr1 = Head
While ptr1 != Null
Set ptr2 = ptr1.next
Whil ptr2 != Null
if ptr1.PRN > ptr2.PRN then
Swap ptr1.PRN and ptr2.PRN
Swap ptr1.Name and ptr2.Name
End if
Set ptr2 = ptr2.next.
End While
Set PTR1 = ptr1.next
End While
Print "List sorted successfully"
Break.

Case 7 :- Reverse List.
Set prev = current
Set current = next
End While.
Set head = current
Print "List reversed successfully"
Break.

Case 8 :- Merge two lists
Create & obtain second list with
head 2.
If head == Null then
Set head = head 2
Else
Set temp = head
While temp.next != Null
Set temp = temp.next.
End while
Set temp.next = head 2
End if
Print "Lists merged successfully"
Break.

Case 9 :-
Print "Exiting program"
Break.
Default :
Print "Invalid choice"

Until Choice == 9
Stop

Time Complexity:-

Operation	Time Complexity
1) Insert at beginning	O(1)
2) Insert at end	O(n)
3) Delete at beginning	O(1)
4) Delete by PRN	O(n)
5) Display all members	O(n)
6) Count nodes	O(n)
7) Search by PRN	O(n)
8) Sort using nested loops	O(n^2)
9) Reverse the list	O(n)
10) Merge two lists	O(n+m)

Conclusion:-
In this practical, a student club enrollment management system was developed using a singly linked list. This program demonstrates dynamic memory allocation and efficient manipulation of linked data structures. Operations such as insertion, deletion, traversal, counting, sorting and reversing were successfully implemented. This practical enhances the understanding of linked list operations and their applications in real-world record management systems.

FAQs:-

Q1 Why is linked list preferred over an array in this application? What information is stored in each node of the circular linked list?
→ It allows dynamic insertion and deletion. Each node stores PRN, name, and a pointer to the next node.

Q2 What is a singly linked list?
→ A list where each node points to the next node.

Q3 What information is stored in each node?
→ PRN, name and next pointer.

Q4 What is dynamic memory allocation?
→ Allocating memory at runtime as needed.

Q5 Which function releases allocated memory?
→ free()

Q6 What is the time complexity of insertion at the beginning?
→ O(1)

Q7 What is the time complexity of searching for a student?
→ O(n)

Q8 How is a linked list reversed?
→ Using three pointers: prev, current, next.

Q9 Which sorting technique can be used in linked list?
→ Bubble sort or merge sort.